What the exam expects: LCD basics • function of RS, R/W, E • 8051–LCD connections • command codes • init + "display 2 strings" program. (Context: LCD = text display topic after LED/ADC/DAC/keyboard in Module 4.)

1 • LCD basics (what it is and why it is used)

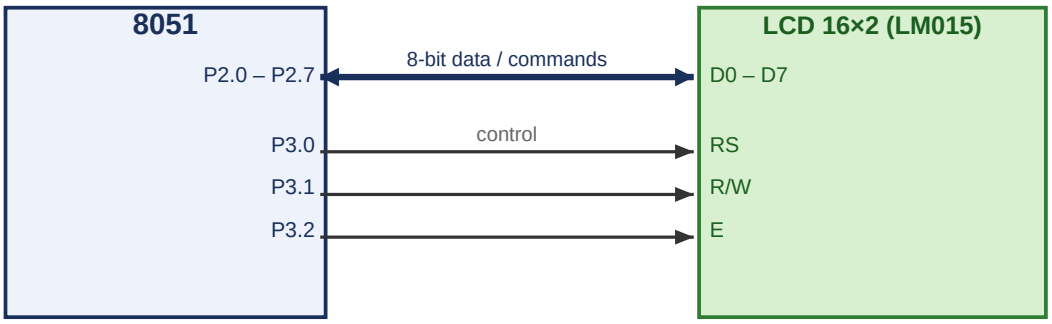
- Liquid Crystal Display — the **cheapest and easiest way to display text** (menus, messages, values).
- Many sizes: **1×20 up to 8×80 characters**; the standard module used here is **16×2** (16 chars × 2 lines, e.g. LM015).
- Has an **integrated controller** — the 8051 only sends commands/bytes; the module forms the characters (5×7 dot matrix).
- The display has **two registers**: a **command register** and a **data register** — **RS** selects which one is used.
- **DB7–DB0** carry **both commands and data**; characters are sent as their **ASCII** codes.

2 • Pin functions (learn this table)

Pin	Full name	Function
VSS / VDD	GND / +5V	Power supply
VEE	Contrast	Contrast control — wiper of a 10K potentiometer
RS	Register Select	RS = 0 → command register • RS = 1 → data register
R/W	Read / Write	R/W = 0 → write (8051 → LCD) • R/W = 1 → read
E	Enable	Latches the data present on the data pins (needs High→Low pulse)
D0–D7	Data pins	8 bidirectional pins — commands and ASCII characters
A / K	Backlight +/–	A → +5V, K → GND (backlight of the module)

3 • Connection to the 8051

- Data bus: LCD **D0–D7** → **Port 2 (P2.0–P2.7)**. Control pins: **RS** → **P3.0, R/W** → **P3.1, E** → **P3.2**.



VSS → GND VDD → +5V VEE → 10K pot (CONTRAST) A → +5V, K → GND (backlight)

All grounds common • RW tied LOW for write-only use in this experiment

Slide variants: the LM015 figure wires RS→P3.3, R/W→P3.4, E→P3.5 with data on P1 — the mapping is not unique. **The program must match the wiring:** change only the EQU lines if your pins differ. Other slides keep R/W = 0 (write-only).

4 • Command codes (sent with RS = 0, R/W = 0)

Code	Function	Note
`38H`	8-bit mode, 2-line LCD (5×7 matrix)	Function set — first command of init
`0CH`	Display ON, cursor OFF	
`01H`	Clear display	Slowest — needs 1.64 ms; clears DD RAM, cursor home
`06H`	Increment cursor	Cursor auto-moves right after each character
`80H`	Force cursor to 1st line, position 1	Also the DD RAM base address of line 1
`C0H`	Force cursor to 2nd line, position 1	DD RAM base address of line 2

Timing (instruction table): every operation ≈ **40 µs** max, except Clear Display / Return Home = **1.64 ms**. Instead of fixed delays you can poll the **Busy Flag**: set RS=0, **R/W=1**, read D7 (D7 = BF) — D7 = 1 means the LCD is still busy.

5 • How ONE byte is sent to the LCD (the pulse recipe)

- **Step 1:** Put the byte on the data pins: `MOV P2,A`.
- **Step 2:** Select the register: `RS = 0` for a command, `RS = 1` for data; keep `R/W = 0` (write).
- **Step 3:** Give the Enable pulse: `EN = 1` → `delay` → `EN = 0` — the **falling edge latches** the byte inside the LCD.
- **Step 4:** Wait (delay / busy-flag poll) so the LCD can execute (≈40 µs, 1.64 ms after 01H).

6 • One-line explanations the examiner looks for

- `EQU` — assigns a pin/port a symbolic name; definitions must match the actual wiring.
- `MOV A,#38H + ACALL LCD_CMD` — load the command in A, send it with RS = 0.
- `MOVC A,@A+DPTR` — reads the next character of the string from code ROM (lookup-table read).
- `JZ` — strings are stored with a **00H terminator**; JZ ends the loop when it is reached.
- `DB 'VIT BHOPAL',00H` — stores ASCII characters in ROM (data directive).
- `SETB / CLR EN` — creates the High→Low edge that latches the byte into the LCD.
- Same **LCD_CMD / LCD_DATA** structure — only the **RS level differs** (0 = command, 1 = data).

Read next page → the complete annotated program (init + display 2 lines) — THE expected answer.

7 • The full program — fully commented (init + display 2 messages)

```
                        ; ---- Pin definitions (match these EQU to your wiring!) ----
RS  EQU P3.0             ; Register Select pin (0 = command, 1 = data)
RW  EQU P3.1             ; Read/Write pin (0 = write to LCD – always write here)
EN  EQU P3.2             ; Enable pin (H-to-L pulse latches the byte into LCD)
        ORG  0000H
MAIN:   ACALL LCD_INIT   ; one-time setup: the 4 commands below
        MOV  A,#80H      ; command 80H: cursor to line 1, position 1
        ACALL LCD_CMD    ; send it as a COMMAND (RS=0)
        MOV  DPTR,#MSG1  ; point to string 1 in ROM
D1_:    CLR  A
        MOVC A,@A+DPTR   ; read character from code ROM
        JZ   LINE2       ; 00H terminator → string finished
        ACALL LCD_DATA   ; send char as DATA (RS=1) – ASCII
        INC  DPTR        ; next character
        SJMP D1_         ; loop till terminator
LINE2:  MOV  A,#0C0H     ; command C0H: cursor to line 2, position 1
        ACALL LCD_CMD
        MOV  DPTR,#MSG2  ; point to string 2
D2_:    CLR  A
        MOVC A,@A+DPTR
        JZ   HERE
        ACALL LCD_DATA
        INC  DPTR
        SJMP D2_
HERE:   SJMP HERE        ; job done – stay here forever
LCD_INIT:                ; -- init: the 4 mandatory commands --
        MOV  P2,#00H     ; clears data port
        MOV  A,#38H      ; 8-bit interface, 2 lines, 5×7 font
        ACALL LCD_CMD
        MOV  A,#0CH      ; display ON, cursor OFF
        ACALL LCD_CMD
        MOV  A,#01H      ; clear display (slowest: 1.64 ms)
        ACALL LCD_CMD
        MOV  A,#06H      ; increment cursor (shift right)
        ACALL LCD_CMD
        RET
LCD_CMD:                 ; -- send COMMAND in A: RS=0, RW=0, pulse E --
        MOV  P2,A        ; put byte on data lines P2
        CLR  RS          ; RS = 0 → LCD treats it as command
        CLR  RW          ; RW = 0 → write direction
        SETB EN          ; E = 1 …
        ACALL DELAY      ; … hold briefly …
        CLR  EN          ; … E = 0: falling edge LATCHES byte
        ACALL DELAY      ; let the LCD process it
        RET
LCD_DATA:                ; -- send DATA/char in A: RS=1, RW=0 --
        MOV  P2,A        ; put character on P2
        SETB RS          ; RS = 1 → goes to DATA register (display)
        CLR  RW          ; write
        SETB EN
        ACALL DELAY
        CLR  EN          ; latch on falling edge
        ACALL DELAY
        RET
DELAY:  MOV  R6,#20      ; -- delay: LCD needs ~40 µs+ per op --
DL1:    MOV  R7,#255
DL2:    DJNZ R7,DL2
        DJNZ R6,DL1
        RET
MSG1:   DB 'VIT BHOPAL',00H  ; strings: ASCII + 00H terminator (line 1)
MSG2:   DB 'Dream to Win',00H displayed on line 2
        END
```

30-second revision: 16×2 LCD, internal controller • RS = command/data select, R/W = 0 write, E latches • D0–D7 → P2, RS/RW/E → P3.0/3.1/3.2 • init = 38H,0CH,01H,06H • 80H/C0H = line 1/2 • CMD vs DATA differ only in RS • strings read via MOVC until 00H.